Appendix 1. Exponential and logarithmic functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.1 Domains of integer powers

For $b \neq 0, n \in \mathbb{Z}_{\geq 0}$, define

$$b^0 = 1, \quad b^n = b \cdot b^{n-1}, \quad b^{-n} = 1/b^n.$$

At $b = 0$, positive integer powers exist, while 0^0 and negative powers require contextual conventions or exclusion. The laws

$$b^{m+n} = b^m b^n, \quad (b^m)^n = b^{mn}, \quad (bc)^n = b^n c^n$$

require all appearing powers to be defined. There is no universal prior evaluation of 0^0; conventions for limits, series, and combinatorial counting are stated where needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.2 Rational powers and principal roots

For $b > 0, p \in \mathbb{Z}, q \in \mathbb{Z}_{>0}$, define $b^{p/q} = (b^{1/q})^p$, where $b^{1/q}$ is the unique positive r satisfying $r^q = b$. Fraction representation does not affect the result, for example $b^{2/4} = b^{1/2}$.

For negative bases, reduce the fraction first. A real rational power is defined through an odd root only when the reduced denominator is odd. Thus $(-8)^{2/6} = (-8)^{1/3} = -2$ is valid despite the unreduced even denominator, while $(-4)^{1/2}$ is not real. The expressions $(x^2)^{1/4}$ and $x^{1/2}$ have different domains; nested-power laws cannot be applied unconditionally to negative bases.

The principal-root convention is part of well-definedness. For $b > 0$, $r^2 = b$ has two real solutions, but its principal square root is the positive one. At zero it is zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.3 Defining exp by a power series

For every real x, define

$$\exp(x) = E(x) = \sum_{n=0}^{\infty} x^n/n!.$$

The ratio test gives infinite radius of convergence and absolute convergence on bounded intervals. Termwise differentiation gives

$$E'(x) = \sum_{n=1}^{\infty} \frac{nx^{n-1}}{n!} = \sum_{m=0}^{\infty} \frac{x^m}{m!} = E(x),$$

with $E(0) = 1$. Thus E solves $y' = y, y(0) = 1$. After establishing the exponential law, any other solution has $(y(x)E(-x))' = 0$ and initial value one, so $y = E$, proving uniqueness. Infinite radius does not make E a polynomial merely because each term is polynomial.

A1.4 Proving the exponential law

Absolute convergence permits multiplication and rearrangement:

$$\begin{aligned} E(x)E(y) &= \sum_{m,k\geq 0} \frac{x^m y^k}{m!k!} \\ &= \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{m=0}^{n} \binom{n}{m} x^m y^{n-m} \\ &= \sum_{n=0}^{\infty} \frac{(x+y)^n}{n!} = E(x+y). \end{aligned}$$

Taking $y = -x$ gives $E(x)E(-x) = 1$, hence nonvanishing. Since $E(x) = E(x/2)^2$, it is positive and $E(-x) = 1/E(x)$.

Define $e = E(1)$. Induction and roots give $E(r) = e^r$ for rational r; continuity extends this to reals. The binomial expansion proves the identity, and absolute convergence on compact intervals justifies rearrangement.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.5 Logarithm as an inverse

Since $E' = E > 0$, E is strictly increasing and injective. For nonnegative x, $E(x) \geq 1 + x$, so it tends to infinity. The reciprocal identity makes $E(x) \to 0$ as $x \to -\infty$. Continuity and the intermediate value theorem therefore give image $(0, \infty)$, and unique inverse functions

$$\exp : \mathbb{R} \to (0, \infty), \qquad \ln : (0, \infty) \to \mathbb{R},$$
$$\ln(e^x) = x, \qquad e^{\ln y} = y.$$

The real logarithm requires positive arguments. Complex logarithms require branch choices beyond this single-valued real definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.6 Logarithm laws and change of base

For $b > 0$, define every real power by $b^r = E(r \ln b)$. Uniqueness of positive roots makes this agree with the earlier rational definition. The exponential law and inverse give, for $a, b > 0, r \in \mathbb{R}$,

$$\ln(ab) = \ln a + \ln b, \quad \ln(a/b) = \ln a - \ln b, \quad \ln(a^r) = r \ln a.$$

They also prove positive-base real-exponent laws:

$$b^{x+y} = E((x+y)\ln b) = b^x b^y,$$

$$(ab)^x = E(x(\ln a + \ln b)) = a^x b^x,$$

$$(b^x)^y = E(y \ln(E(x \ln b))) = E(xy \ln b) = b^{xy}.$$

The quotient law follows from $b^{-x} = 1/b^x$. All require positive bases and the established inverse.

For $b > 0, b \neq 1$, define $\log_b x = \ln x / \ln b$. Fixed $a, b > 1$ give

$$\log_a x = \frac{\log_b x}{\log_b a},$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.6 Logarithm laws and change of base (continued)

a positive constant factor, hence Θ-equivalence. For a base between zero and one, logarithms are negative at $x > 1$, so positive-cost comparison cannot use the same positive-factor argument directly. Compare absolute values, $|\log_a x| = \Theta(|\log_b x|)$, or retain signs explicitly.

Equal asymptotic classes do not imply equal implementation cost, error, or overflow behavior. Also $\ln(a + b) \neq \ln a + \ln b$; the law concerns products.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.7 Derivatives, growth, and boundaries

The inverse derivative gives $(\ln x)' = 1/x$, and $(b^x)' = (\ln b)b^x$. For every real x, $e^x \geq 1 + x$. Nonnegative x permits stronger polynomial bounds from the series. In particular, for fixed n, $x^n/e^x \to 0$, so $x^n = o(e^x)$.

For $b = 0.5$, b^x decreases; it increases only for $b > 1$. State the base range. Mathematical identities need not behave equally in floating point. With $a = b = 10^{200}$, double-precision ab overflows while $\log(a) + \log(b)$ can remain finite. Numerical strategy depends on the input range.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.8 Existence and representation independence of rational powers

For $b > 0$ and positive integer q, the function t^q is continuous and strictly increasing on nonnegative reals. For $t > s \geq 0$,

$$t^q - s^q = (t - s)(t^{q-1} + \cdots + s^{q-1}) > 0.$$

The intermediate value theorem between zero and sufficiently large t gives a unique positive q-th root of b.

If $p/q = p'/q'$, raising the two positive candidates to power qq' gives $b^{pq'} = b^{p'q}$. Injectivity of a positive integer power forces equality. Reciprocals handle negative numerators. Thus the definition is independent of fractional representation.

For negative bases, even defined nested powers may fail the usual law:

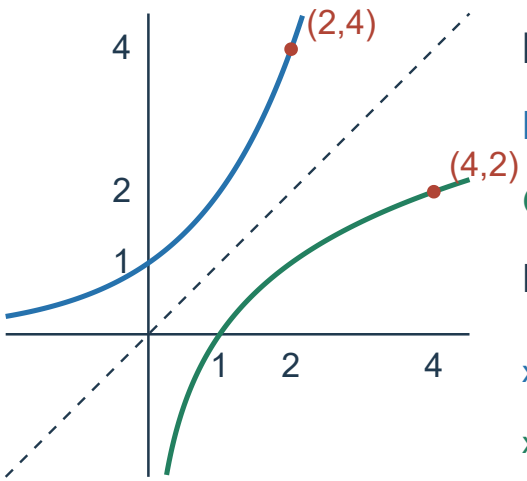
$$((-8)^2)^{1/2} = 8, \qquad (-8)^{2(1/2)} = -8.$$

Existence of both expressions alone does not extend the positive-base law.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.9 Inverse graphs and base-dependent monotonicity

Inversion swaps coordinates across y=x; the base determines monotonicity



b=2

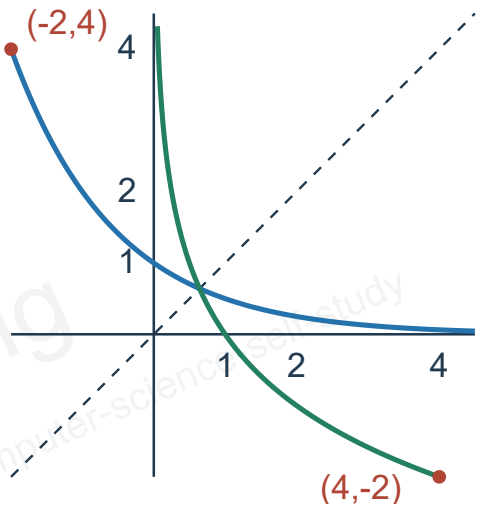
Blue: y=bˣ

Green: y=log_b x

Dashed: y=x

x∈ℝ → y>0

x>0 → y∈ℝ



b=0.5

Blue: y=bˣ

Green: y=log_b x

Dashed: y=x

x∈ℝ → y>0

x>0 → y∈ℝ

Inversion exchanges (x, b^x) and (b^x, x), reflecting the graph in $y = x$. Both functions increase for $b > 1$, where $\ln b > 0$, and decrease for $0 < b < 1$, where $\ln b < 0$. Both remain one-to-one. The exponential range and logarithm domain are the positive reals.

Because the base is below one,

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.9 Inverse graphs and base-dependent monotonicity (continued)

$$\log_{1/2} x \geq 2 \iff 0 < x \leq (1/2)^2 = 1/4.$$

The inverse reverses the inequality. Substituting $x = 1/8$ gives $3 \geq 2$, while $x = 1$ gives zero and fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.10 Quantified growth proofs

For fixed integer $k \geq 0, x > 0$, one series term gives

$$e^x \geq \frac{x^{k+1}}{(k+1)!} \Rightarrow 0 \leq \frac{x^k}{e^x} \leq \frac{(k+1)!}{x} \to 0.$$

For fixed real $r \geq 0$, choose integer $k \geq r$; when $x \geq 1$, $x^r \leq x^k$, proving $x^r = o(e^x)$. For fixed $b > 1$, substituting $t = x \ln b$ gives $x^r = o(b^x)$.

For fixed $\varepsilon > 0$, put $u = \varepsilon \ln x \to \infty$. Then

$$\frac{\ln x}{x^\varepsilon} = \frac{1}{\varepsilon} \frac{u}{e^u} \to 0.$$

Thus $\ln x = o(x^\varepsilon)$. Fixing ε, r, b matters; the same conclusion need not hold if b approaches one with x.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.11 Halving rounds, bit lengths, and rounding

A positive integer's binary length is $\lfloor \log_2 n \rfloor + 1$. The unique k satisfying $2^k \leq n < 2^{k+1}$ indexes its highest significant bit.

n	Bounding powers	Binary	Length
8	$8 \leq 8 < 16$	1000	4
13	$8 \leq 13 < 16$	1101	4
16	$16 \leq 16 < 32$	10000	5

Replacing workload n by $\lceil n/2 \rceil$ each round until one needs exactly $\lceil \log_2 n \rceil$ rounds for $n \geq 1$, zero at one. After t rounds the value is $\lceil n/2^t \rceil$, by induction using $\lceil \lceil z \rceil / 2 \rceil = \lceil z/2 \rceil$. It first reaches one iff $2^t \geq n$. This differs from bit length, especially at powers of two.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.12 Logarithms within numerical range

For $L = \ln(e^a + e^b)$, set $m = \max(a, b)$:

$$L = m + \ln(e^{a-m} + e^{b-m}).$$

Both exponents are nonpositive and one exponential equals one. The sum lies in $[1, 2]$, avoiding overflow in e^a. For $a = 1000, b = 999$, $L = 1000 + \ln(1 + e^{-1}) \approx 1000.313262$. This applies a real identity with input-range control, not the false splitting of a logarithm of a sum.

For the exponential series on $|x| \leq M$, $|x^n/n!| \leq M^n/n!$, with a convergent majorant. The Weierstrass M-test gives uniform convergence. The derivative series has the same bounded-interval estimate. Termwise differentiation uses a power-series theorem, not merely differentiability of individual terms.

Separate exercise and solution PDFs provide domain checks, counterexamples, and quantitative bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study